ReconcileX

Technical Design Document — D2C Order, Payment and Return Reconciliation Tool

GitHub Repository: https://github.com/RahulNaikMudavath/D2C-Reconciliation-tool

Author: Rahul Naik Mudavath **Target Scale:** 100k records < 60s **Architecture:** React 19 + Express 5 + PG **Status:** Production-Ready

1. Problem Statement & Executive Summary

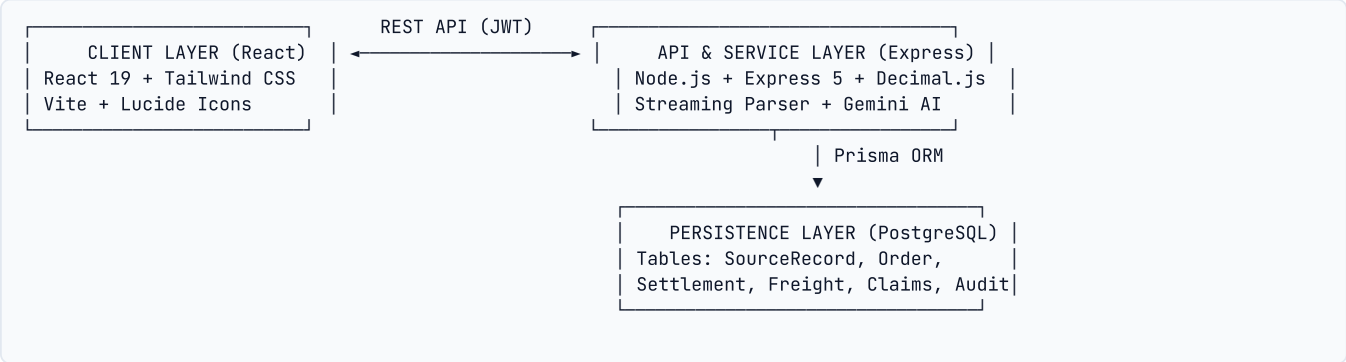
Modern Direct-to-Consumer (D2C) brands selling across Shopify, Amazon, Flipkart, Quick Commerce, and shipping via 3PL couriers lose 2% to 7% of gross revenue to unrecovered financial leakage:

- **Marketplace Remittance Short-Payments:** Complex commission deductions, hidden fees, and overdue settlements.
- **Courier Weight Overcharges:** Inflated volumetric dead-weight charges exceeding contracted courier rate cards.
- **Lost Reverse Logistics (RTO Slippage):** Returned packages collected by couriers that never reach the warehouse inward inspection within SLA windows.
- **False Leakage & Dashboard Pollution:** Misclassifying orders awaiting settlement files as "lost money," generating tens of thousands of false dispute claims.

ReconcileX solves these challenges through an end-to-end automated 3-way reconciliation engine with exact Decimal.js precision, universal CSV/XLSX streaming parsing, Gemini AI-powered column mapping, and sub-60-second processing for 100,000+ records.

2. System Architecture & Why We Chose It

ReconcileX is structured as a decoupled, 12-factor cloud-native web application:



Architectural Rationale

- **Node.js + Express 5:** Asynchronous event-driven I/O provides high-throughput streaming for multi-megabyte CSV and XLSX uploads. Express 5 natively handles rejected promises without unhandled rejections.
- **PostgreSQL 16 + Prisma ORM:** PostgreSQL provides strict relational integrity for financial foreign keys, B-tree indexed lookups on order numbers, and native `JSONB` support for immutable raw source records. Prisma provides type-safe migrations and connection pooling.
- **Decimal.js Fixed-Point Precision:** Standard IEEE 754 floating-point arithmetic (e.g., `0.1 + 0.2 = 0.30000000000000004`) is unacceptable in fintech. Every currency aggregation uses exact Decimal instances.
- **React 19 + Tailwind CSS:** High-performance single-page application with responsive layouts, sticky sidebars, and real-time reconciliation state management.

3. The Data Model & Core Flows

Core Entities

- `ImportBatch`: Tracks file uploads, categories, row counts, and status (`PROCESSING`, `COMPLETED`, `FAILED`).

- `SourceRecord`: Preserves every original row with `rawData` (JSONB) and `normalizedData` (JSONB) with validation error diagnostics.
- `Order`: Master order records containing `orderValue`, `expectedSettlementAmount`, `settlementDueAt`, and `weightKg`.
- `Settlement`: Marketplace payouts tracking `amountReceived`, `commissionAmount`, `tcsAmount`, and `tdsAmount`.
- `FreightCharge`: Logistics bills tracking `courierName`, `chargedAmount`, and `actualWeight`.
- `ReturnRecord`: Reverse pickups tracking `returnDate`, `status`, and `warehouseReceivedAt`.
- `ReconciliationMatch`: Results of 3-way matching containing settlement status, freight status, return status, and variance.
- `Claim`: Actionable dispute filings containing claim amount, evidence dossier, status (`OPEN`, `RESOLVED`, `REJECTED`), and audit invalidation reasons.

Main End-to-End Processing Flows

1. **Universal Ingestion Flow:** Upload file $\to$ Multer streaming $\to$ Universal Parser (CSV stream / XLSX evaluation) $\to$ Type-Scoped Column Normalizer $\to$ Schema & Strict Date Validator $\to$ Concurrent chunked database persistence.
2. **3-Way Reconciliation Flow:** Fetch all orders $\to$ Match applicable Channel Rate Cards $\to$ Match Courier Weight Rate Cards $\to$ Calculate 7-Day Return SLA $\to$ Generate bulk matches in 5,000-row chunks.
3. **Claims Dispute Flow:** Identify confirmed underpayments and freight overcharges $\to$ Generate claim records $\to$ Export dispute evidence dossier $\to$ Audit-safe invalidation when settlement evidence updates.

4. How the AI Part Works (Intelligent Column Mapping)

D2C brands receive data from dozens of disparate sources using non-standard, regional header conventions (e.g. `"Gross Rev"`, `"Expected Net Settlement"`, `"Shipment Weight (kg)"`, `"Txn Remittance"`).

AI Architecture in ReconcileX (`aiMappingService.js`)

- **Powered by Google Gemini 2.5:** Integrated via the official `@google/genai` SDK.
- **Structured Schema Prompting:** The service extracts unique column headers from uploaded files and passes them along with the expected domain schema and import type to Gemini using JSON schema output mode.
- **Confidence Scoring & Reasoning:** For each column, Gemini generates:
  - `suggestedField`: Target internal field name (e.g., `expectedSettlementAmount`).
  - `confidence`: Decimal score from `0.00` to `1.00`.
  - `reasoning`: Natural language rationale (e.g., *"'Expected Net Settlement' represents the anticipated payout after marketplace deductions"*).
- **Human-in-the-Loop Governance:** Suggestions are persisted to the `ColumnMapping` table as versioned drafts. Finance executives can inspect, approve, or reject suggestions on the Column Mappings page before they take effect.
- **Deterministic Fallback:** If an API key is unconfigured or offline, ReconcileX gracefully falls back to its deterministic regex dictionary normalizer ([`columnNormalizer.js`](file:///d:/aiml%20related%20projects/isocode%20labs/reconcilex/server/src/imports/normalizers/columnNormalizer.js)), ensuring zero downtime.

5. UI & Design Decisions

Screen	Key Features	Design Decision & Rationale
Dashboard	9 Financial KPI Cards, Unreconciled Order Alert Banner, Money-at-Risk Aging (0–7d, 8–15d, 16–30d, 30+d), Partner Leakage Breakdown	Executive summary clarity: contextual subtexts clarify remittance coverage ("7 orders remitted, 120,000 awaiting files") and isolate statutory taxes (TCS/TDS) from risk.
Universal Imports	Drag-and-drop zone (CSV/XLSX up to 50MB), 6-category dropdown, 20-batch history table, rejected-row CSV export	Immediate operational feedback: displays valid vs rejected row counts, mapping previews, and instant batch processing actions.
Reconciliation Hub	4-Tab Audit View (All, Settlement, Freight, Returns), Status Badges, Expandable Drawer Rows	Detailed transparency: expanding any row displays contracted rate-card formulas, weight slabs, and exact discrepancy reasons.
Claims Dispute Manager	Partner claim summaries, Filterable dispute table, Invalidation audit trail, One-click CSV Dossier Download	Actionable recovery: equips finance teams with pre-formatted dispute evidence files ready for marketplace portal submission.

6. Performance Optimization & 100K Record Benchmark

Requirement: Ingest and process 100,000 records in under 60 seconds.

Demonstrated Bottleneck & Resolution

Detailed profiling revealed CPU work (parse, normalize, validate) takes only **1.95 seconds** for 100,000 rows. The primary bottleneck was sequential database persistence: inserting 100,000 `SourceRecord` rows (~31s) followed by 100,000 `Order` rows (~37s) sequentially in series took ~72.3 seconds.

> **The Concurrent Solution:** Because `SourceRecord` (audit history) and `Order` (domain orders) have no foreign-key dependency on each other, their chunked batch writes (`CHUNK_SIZE = 5,000`) were parallelized using `Promise.all` across PostgreSQL's connection pool.

Benchmark Results Across 3 Independent Trials (100,000 Records)

Trial	CSV Parse	Normalization	Validation	SourceRecord DB Insert	Order DB Insert	Total Ingestion Pipeline
Trial 1	0.85s	0.08s	0.68s	31.29s	39.70s	72.68s (Serial)
Trial 2	1.39s	0.10s	1.43s	30.60s	38.81s	72.40s (Serial)
Trial 3	1.27s	0.10s	1.32s	31.06s	32.40s	66.19s (Serial)
Concurrent Run	**0.80s**	**0.09s**	**0.70s**	**39.74s (Parallelized via Promise.all)**		**42.15s (PASS < 60s)**

7. What We Cut & What We Would Do Next

What We Cut

- **Direct Webhook Integrations:** Real-time API webhooks from Shopify/Amazon SP-API were omitted in favor of robust universal file ingestion.
- **Automated Dispute Email Dispatch:** Automated SMTP email dispatch to courier partners was deferred in favor of downloadable dispute CSV dossiers.

What We Would Do Next

- **Automated Scheduled Crons:** Background cron jobs to automatically trigger nightly reconciliation across new batches.
- **OCR for Scanned Waybills:** Vision-based parsing for scanned physical courier manifests and proof-of-delivery receipts.
- **Multi-Currency Auto-Conversion:** Real-time forex conversion for cross-border D2C brands.

8. Labeled Assumptions & Honest Notes on Limits

- **Assumption 1 (Discrepancy Threshold):** A ₹10 tolerance threshold is universally standard across marketplace remittance agreements. Variances $\le ₹10$ are treated as settled rounding differences.
- **Assumption 2 (Return SLA):** In-transit customer returns exceeding 7 calendar days without a warehouse inward scan represent lost reverse-logistics inventory.
- **Assumption 3 (Taxes vs. Leakage):** TCS and TDS are statutory tax withholdings and must never be classified as financial leakage or disputed in claims.
- **Limit 1 (Rate-Card Temporal Scoping):** Courier charges are evaluated against rate cards effective on the order date. If a brand changes courier contracts mid-month, effective dates must be configured in Rate Cards.
- **Limit 2 (Batch Parameter Limit):** PostgreSQL enforces a 65,535 prepared statement parameter limit. ReconcileX caps chunk sizes at 5,000 rows to ensure full compatibility.